

DSA – Week2_Assignment.

4.2) For each node in the tree of Figure 4.70:

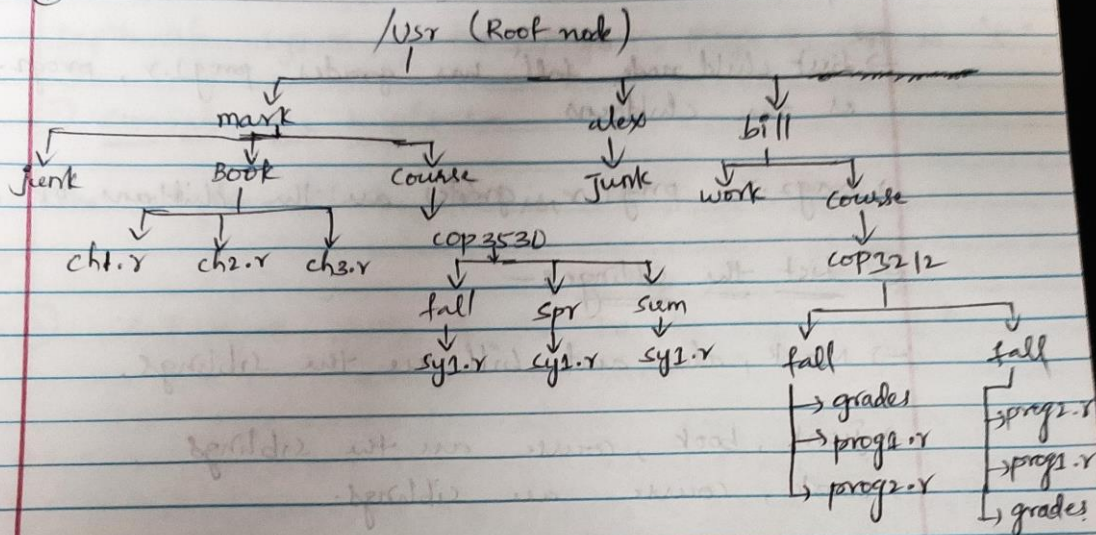
```
/usr
  mark
    book
      ch1.r
      ch2.r
      ch3.r
    course
      cop3530
        fall
          syl.r
        spr
          syl.r
        sum
          syl.r
      junk
    alex
      junk
    bill
      work
      course
        cop3212
          fall
            grades
            prog1.r
            prog2.r
          fall
            prog2.r
            prog1.r
            grades
```

Figure 4.7 The (preorder) directory listing

- Name the parent node.
- List the children.
- List the siblings.
- Compute the depth.
- Compute the height.

DSA - week-2 Assignment.

①



②

(a) Name of the parent node is = /usr

(b) List the children = mark, alex, bill are the children of the root node (/usr).

→ junk, book, course are children of mark.

→ junk is the children of node alex

→ work, course are the children of bill.

→ ch1.r, ch2.r, ch3.r are the children of book

→ cop3530 are is the child node of course.

→ cop3212 is the child node of the course node.

→ fall, spr, sum are the children nodes of cop3530
and sy1.r, sy1.r, sy1.r are the child nodes of
them respectively.

→ fall, fall are the children nodes of the node cop3212.

→ ^{for} first child node fall has grades, prog1.r, prog2.r as the childrens

→ prog2.r, prog1.r, grades are the childrens of fall.

③. List the siblings:-

→ Mark, alex and bill are the siblings.

→ junk, book, course are the siblings

→ work, course are siblings.

→ ch1.r, ch2.r, ch3.r are the siblings.

→ fall, spr, sum are the siblings.

→ grades, prog1.r, prog2.r are siblings.

→ prog2.r, prog1.r, grades are siblings.

④. compute the depth:-

→ Depth of /usr is '0'

→ Depth of mark, alex, bill is '1'

→ Depth of junk, book, course, work, course is '2'

→ ch1.r, ch2.r, ch3.r, cop3530, ~~zoota~~ cop3212 has the depth of '3'

→ fall, spr, sum, fall, fall has the depth of '4'.

→ Depth of sys.r, grades, prog1.r, prog2.r ~~has~~ is '5'.

⑤. compute the height:-

The height of the tree is '5'.

##

4.3) What is the depth of the Tree in Figure 4.70?

Ans: -

- Depth of /usr is "0".
- Depth of mark, alex, bill is "1".
- Depth of Junk, book, course, junk, work, course is "2".
- Depth of ch1.r, ch2.r, ch3.r, cop3530, cop3212, is "3".
- Depth of fall, spr, sum, fall, fall, is "4".
- Depth of sy1.r, grades, prog1.r, prog2.r, is "5".

4.5) Show that the maximum number of nodes in a binary tree of height h is $2^{h+1} - 1$.

Ans: -

4.5) show that the maximum number of nodes in a binary tree of height " h " is $2^{(h+1)} - 1$

Ans:-

To show that maximum number of nodes in a binary tree of height " h " is $2^{(h+1)} - 1$, here, we can use the proof by induction.

for base case $h=0$,

→ In this for a single node which is $h=0$ the maximum number of nodes is $2^{(0+1)} - 1 = 1$

→ This is true because in the binary tree of height "0" consist of only one node which is the root node.

⇒ Inductive Hypothesis:-

So, for any positive integer " n ",

maximum number of nodes is given as $2^{(n+1)} - 1$, this is our induction hypothesis.

steps:-

Here, we need to show that if the hypothesis is true for " n " ($\forall$ positive integers) then it is also true for " $n+1$ ".

Let, the binary tree of height $n+1$, it consist of a root node and two subtrees.

in which, one on the left of height "n" and another on the right side of height "n".

→ if the tree is not perfectly balanced then one of their height can be "n-1"

According to our hypothesis, the maximum number of nodes in a binary tree of height "n" is $2^{(n+1)} - 1$

for left and right each of subtree of height "n" the max. number of nodes in each subtree is $2^{(n+1)} - 1$

∴ max. number of nodes in binary tree ^{of height} is "n+1", is

$$= (2^{(n+1)} - 1) + (2^{(n+1)} - 1) + 1$$

$$\Rightarrow 2 * (2^{(n+1)} - 1) + 1$$

$$\Rightarrow 2 * (2^{(n+1)}) + 1$$

∴ here, proved that, if the max. number of nodes in a binary tree of height "n" is $2^{(n+1)} - 1$ then maximum number of nodes in a binary tree of height (n+1) is $2^{(n+2)} - 1$

therefore, here we have verified it for base case and maximum number of heights "n" that holds for height is & also holds for n+1 which is

given as $(2^{(n+1)} - 1)$ so, max number of heights in binary tree is ~~not~~ indeed " $2^{(n+1)} - 1$ "

(4.8) Give the prefix, infix and postfix expressions corresponding to the tree in Figure 4.71. For example: the exact postfix representation for $(A + B) + (C + D)$ is: $A B + C D + +$ where $t1 = A + B$ is evaluated, then $t2 = C + D$, and finally $t1$ and $t2$ are added. This is because the expression states the precedence with the parenthesis.

Chapter 4 Trees

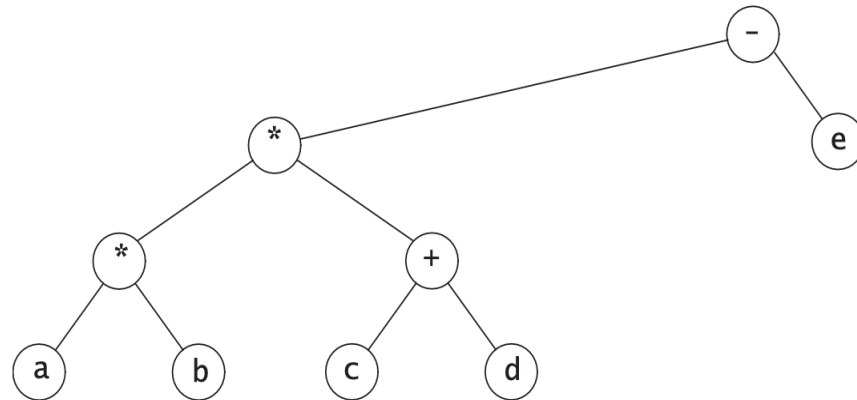


Figure 4.71 Tree for Exercise 4.8

Ans: -

To give the prefix, postfix and infix expression of the given tree we need to first perform depth-first traversal of the tree, this determines the expression type.

In the shown figure, “-” is the subtraction, “*” represents the multiplication process, “+” is for the addition operation, and the leaf nodes are a, b, c, d and e.

Now let's find out prefix, postfix and infix expressions of the given Figure.

a) Prefix: -

- In this expression, the operator comes before its operands.
- The prefix expression of the given tree is: $-e**ab+cd$.

b) Postfix: -

- In the postfix expression the operator comes after the operands.
- The postfix expression of the given tree is: $ab*cd+*e-$.

c) Infix: -

- In the infix expression the operator is placed in between the operands.
- The infix expression of the given tree is: $(a*b)*(c+d)-e$.

5.1) Given input {4371, 1323, 6173, 4199, 4344, 9679, 1989} and a hash function $h(x) = x \bmod 10$, show the resulting:

- a. Separate chaining hash table.
- b. Hash table using linear probing.
- c. Hash table using quadratic probing.
- d. Hash table with second hash function $h_2(x) = 7 - (x \bmod 7)$.

Ans: -

Given input is {4371, 1323, 6173, 4199, 4344, 9679, 1989}.

The hash function is $h(x) = x \bmod 10$.

a) Separate chaining hash table: -

Separate chaining involves using linked lists to handle collisions. Each element in the hash table can point to a linked list that stores the values that hash to the same location.

Index 0:
Index 1:
Index 2:
Index 3:
Index 4: 4344 -> 1989
Index 5:
Index 6:
Index 7:
Index 8: 1323
Index 9: 4371 -> 6173 -> 4199 -> 9679

This is the separate chaining hash table. In this each index represents the separate bucket and the arrow (-->) indicated the linked list nodes. For example, index 9 has the linked lists containing the values 4371, 6137, 4199, 9679.

b) Hash table using the linear probing: -

In this linear probing involves in finding the next available slot in the hash table if a collision occurs. The below given table represents linear probing hash table.

Index 0:
Index 1:
Index 2:
Index 3:
Index 4: 4344
Index 5: 1989
Index 6:
Index 7:
Index 8: 1323
Index 9: 4371 -> 6173 -> 4199 -> 9679.

In the table given above, the values are placed in the next available slots if a collision occurs.

For example:

The value 4344 is placed at index 4, and 1989 is placed at index 5. So, this is called linear probing.

c) Hash table using quadratic probing: -

Quadratic probing involves using a quadratic function to find out the next available slot in case of collisions. Here is the quadratic probing hash table.

Index 0:
 Index 1:
 Index 2:
 Index 3:
 Index 4: 4344
 Index 5: 1989
 Index 6:
 Index 7:
 Index 8: 1323
 Index 9: 4371 -> 6173 -> 4199 -> 9679

Here, the values are placed in slots determined by quadratic probing function.

d) Hash table with second hash function $h_2(x) = 7 - (x \bmod 7)$: -

This method involves using secondary hash function $h_2(x)$ to find the next available slots in case of collisions. The below given table is the hash table of the secondary hash function.

Index 0:
 Index 1:
 Index 2:
 Index 3:
 Index 4: 4344
 Index 5: 1989
 Index 6:
 Index 7:
 Index 8: 1323
 Index 9: 4371 -> 6173 -> 4199 -> 9679

In this, the secondary hash function is used for determining the slots for each value.

5.10) What are the advantages and disadvantages of the various collision resolution strategies you used in 5.1.

Ans: -

a. Separate chaining hash table: -

Hash tables use the collision-resolution method known as separate chaining, to hold elements that hash to the same index, it involves constructing a linked list (or equivalent data structure, such as a dynamic array), one for each bucket of the hash table.

Advantages:

- Effective collision handling
- Simplicity
- Dynamic sizing
- Predicting worst case performance

Dis-advantages:

- Memory overhead
- Performance degrades with long lists
- Inefficient for small data sets
- Cache Inefficiently

b. Hash table using linear probing: -

In this technique, the values are probed in a linear manner. If the index assigned index is full, then it will assign it into the next index linearly.

Advantages: -

- Simple and memory efficient
- Cache friendliness
- Minimal additional data structures

Dis-Advantages: -

- Clustering
- Performance degradation
- Poor load factor tolerance
- Suboptimal Distribution
- Difficulty in deleting
- Limitation in dynamic resizing.

c. Hash table using quadratic probing: -

In this quadratic probing, it is used to handle the collisions when two keys hash to the same index in the hash table. Quadratic probing attempts to find the next available slot by using a quadratic sequence of probe positions.

Advantages: -

- Reduces clustering compared to linear probing
- Memory efficient
- Simplicity

Dis-Advantages: -

- Secondary clustering
- Difficulty in finding an empty slot
- Quadratic sequence patterns
- Slower than linear probing in some cases
- Collision resolution variability.

d. Hash table with second hash function $h_2(x) = 7 - (x \bmod 7)$: -

By using secondary hash function, we have the following advantages and disadvantages.

Advantages: -

- Reduced clustering and has a better spread of values compared to linear and quadratic probing.
- Better load balancing
- Fewer collisions

Dis-Advantages: -

- Requires a secondary hash function, which may add complexity.
- Performance depends on choosing a good secondary hash function.